Team Contribution Document

Project: Hybrid Automation Framework (UI + API)
Application Under Test: https://automationexercise.com
Team Size: 4
Framework Type: Multi-module Maven (Common, UIAutomation, APIAutomation)

Team Member	Email	Phone
Bandi Poorna Sri Sailaja	sailubandi33@gmail.com	6300655517
Andra Tulasi Lakshmi Tanya	tanyakonapala@gmail.com	6302601497
Nerella N S Venkata Siva Sai Rihitha	rihitha9@gmail.com	9290069793
Rambha Rasmitha Lekha	rambhasrasmi7024@gmail.com	91214 87799

Project Overview

The project integrates **Web UI Automation** and **API Automation** under a unified hybrid framework. Each module contributes to a scalable, maintainable test architecture with shared utilities in the **Common** folder.

Folder Structure Summary

Module	Purpose	Key Components
Common/	Shared utilities and configuration	ConfigReader, FrameworkConstants, DriverFactory, LogManager, WaitUtils
UIAutomation/	Web UI testing using Selenium + Cucumber	PageObjects, StepDefinitions, Feature files, Reports, Screenshots
APIAutomation/	API testing using RestAssured + Cucumber	ApiClient, RequestBuilder, Validators, Schemas, Allure Reports

Team Members Contributions

Framework Architect

Primary Focus: *Common Module & Project Integration*

All team members contributed equally

- Designed the **multi-module Maven architecture** ensuring seamless integration between UI and API modules.
- Developed **ConfigReader**, **DriverFactory**, and **FrameworkConstants** to centralize configuration and eliminate hardcoding.

- Implemented **LogManager** and **WaitUtils** for consistent logging and synchronization across modules.
- Managed **pom.xml** dependencies and ensured compatibility between Selenium, RestAssured, and TestNG.
- Oversaw **parallel execution setup** and **cross-module reporting**.

UI Automation

Primary Focus: *UIAutomation Module*

All team members contributed equally

- Implemented **Page Object Model (POM)** for all major pages: Login, Register, Cart, Product, and Account.
- Developed **Step Definitions** for Cucumber scenarios (TC1–TC5) ensuring no Selenium commands inside steps.
- Created **Feature files** for registration, login, cart operations, and validations.
- Integrated **Extent Reports** and **log4j2.xml** for structured reporting and logging.
- Configured **automatic screenshot capture** on test failure.
- Validated UI flow using **TestRunner** and maintained **testing_ui.xml** for execution control.

API Automation

Primary Focus: *APIAutomation Module*

All team members contributed equally

- Built **layered API design**: Client → RequestBuilder → Validator.
- Developed **step definitions** for API test cases (TC1–TC6) using RestAssured.
- Implemented **JSON schema validation** for response integrity.
- Created **UserDataFactory** and **POJO classes** for dynamic payload generation.
- Configured **Allure Reports** for API test visualization.
- Ensured **error handling**, **negative validation**, and **response logging**.

Common Folder & Documentation

Primary Focus: *Hybrid Integration & Reporting*

All team members contributed equally

- Integrated **UI and API modules** using shared **Common utilities**.
- Verified **data consistency** between UI and API layers (e.g., user creation and validation).
- Managed **testng.xml** for unified execution and parallel runs.
- Consolidated **reports, logs, and screenshots** from both modules.
- Authored **README.md**, **architecture documentation**, and this **team contribution document**.
- Ensured adherence to **coding standards**, **naming conventions**, and **Git version control**.

Collaboration Highlights

- **Version Control:** All members contributed via Git branches and pull requests.
- **Code Reviews:** Conducted weekly peer reviews to maintain clean, reusable code.
- **Testing Strategy:** Combined BDD (Cucumber) and TDD (TestNG) for comprehensive coverage.
- **Reporting:** Unified reporting structure for UI and API results under /reports and /logs.

Outcome

The team successfully delivered a **robust, modular, and scalable hybrid automation framework** that:

- Supports **parallel execution** and **cross-layer validation**.
- Produces **centralized reports and logs**.
- Demonstrates **clean architecture** and **reusability** across UI and API layers.